Devoir d'application

Authentification et autorisation avec Spring Security, JWT et React

Technologies

Spring Boot – Spring Security – JWT – React – Axios – JPA – H2/MySQL

---

1. Contexte

Vous devez développer une application web permettant à des utilisateurs d'une entreprise de déclarer et de suivre des incidents informatiques ou de cybersécurité.

Exemples d'incidents :

- ordinateur infecté par un malware ;
- tentative de phishing ;
- problème d'accès à un compte ;
- panne réseau ;
- fuite potentielle de données ;
- problème logiciel ou matériel ;
- comportement suspect sur un poste de travail.

L'application doit gérer trois types d'utilisateurs :

- **EMPLOYEE** : déclare et consulte ses propres incidents ;
- **TECHNICIAN** : traite les incidents qui lui sont affectés ;
- **ADMIN** : gère les utilisateurs et affecte les incidents aux techniciens.

Objectifs

À travers ce devoir, vous devez réutiliser les notions étudiées dans l'atelier JWT et compléter votre solution par plusieurs mécanismes de sécurité et de contrôle d'accès.
Vous devez notamment mettre en place :

- l'authentification avec JWT ;
- la gestion des rôles ;
- la protection des routes backend ;
- la protection des routes React ;
- la gestion des droits d'accès aux données ;
- la validation des données ;
- la gestion des erreurs ;
- un tableau de bord ;
- le suivi des incidents.

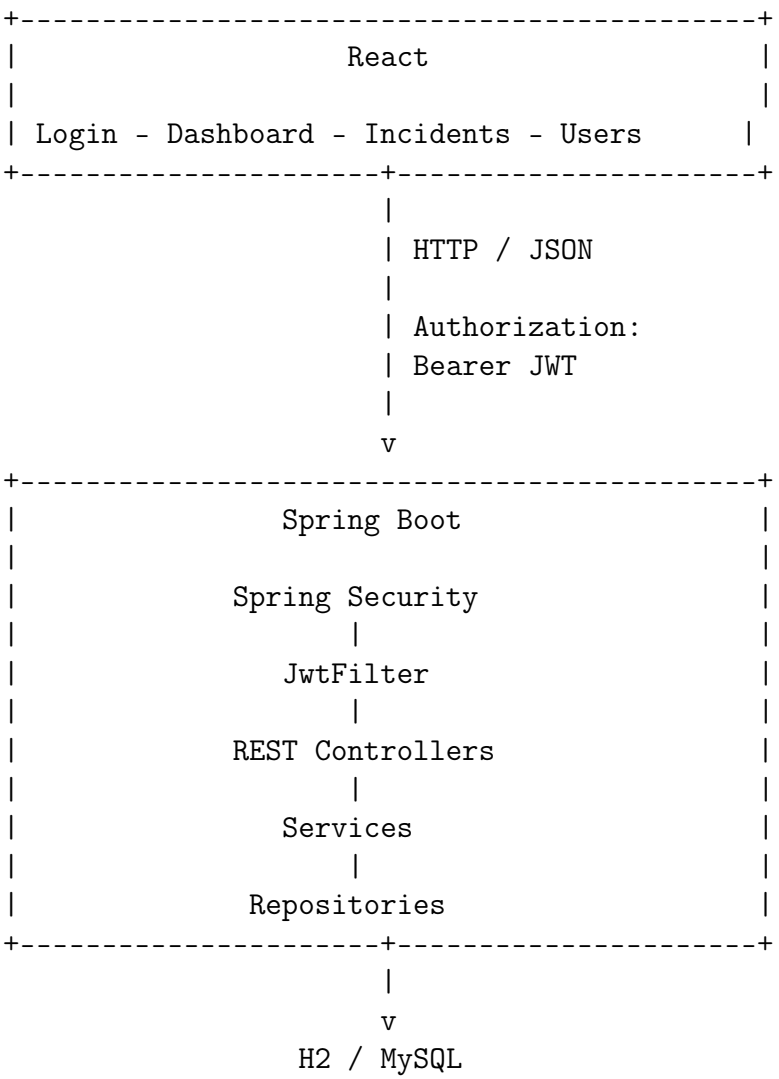
2. Architecture générale

Vous devez créer deux projets :

— un projet backend Spring Boot ;

— un projet frontend React.

L'architecture générale sera :



3. Modèle de données

Vous devez créer au minimum les entités suivantes.

3.1. Utilisateur

Créer une entité `User` contenant :

— id ;

— fullName ;

— email ;

— password ;

— role ;
— enabled ;
— createdAt.

Les rôles disponibles sont :

```
EMPLOYEE
TECHNICIAN
ADMIN
```

Le champ `enabled` doit permettre d'activer ou désactiver un compte.

3.2. Incident

Créer une entité `Incident` contenant au minimum :

— id ;
— title ;
— description ;
— category ;
— priority ;
— status ;
— createdAt ;
— updatedAt ;
— createdBy ;
— assignedTo.

3.3. Catégories

Utiliser par exemple :

```
NETWORK
MALWARE
PHISHING
ACCESS
SOFTWARE
HARDWARE
DATA_LEAK
OTHER
```

3.4. Priorités

```
LOW
MEDIUM
HIGH
CRITICAL
```

3.5. États

```
OPEN
ASSIGNED
IN_PROGRESS
RESOLVED
CLOSED
```

4. Relations entre les entités

Un incident doit être associé :

— à l'utilisateur qui l'a créé ;

— éventuellement au technicien chargé de le traiter.

Travail de recherche

Recherchez comment modéliser les relations entre `User` et `Incident` avec JPA.
Étudiez notamment :

```
@ManyToOne
@OneToMany
LAZY
EAGER
```

Justifiez vos choix.

5. Authentification

Vous devez créer les endpoints publics suivants :

Méthode	Endpoint	Description
POST	/auth/register	Créer un compte
POST	/auth/login	Se connecter

Lors de l'inscription, le rôle attribué automatiquement doit être :

```
EMPLOYEE
```

L'utilisateur ne doit pas pouvoir choisir lui-même son rôle.
Lors d'une connexion réussie, le backend doit retourner au minimum :

```
{
    "token": "eyJhbGciOiJIUzI1NiJ9...",
    "role": "EMPLOYEE",
    "fullName": "Ahmed Benali"
}
```

6. JWT

Le JWT doit contenir :

— l'email de l'utilisateur ;
— la date de création ;
— la date d'expiration ;
— le rôle de l'utilisateur.

Travail de recherche

Recherchez :

— ce qu'est un claim JWT ;
— comment ajouter un rôle dans le token ;
— comment récupérer un claim ;
— comment vérifier l'expiration du token.

7. Configuration Spring Security

Configurez Spring Security afin d'obtenir le comportement suivant.

7.1. Routes publiques

```
/auth/**
/h2-console/**
```

7.2. Routes protégées

```
/api/**
```

L'application doit utiliser :

```
SessionCreationPolicy.STATELESS
```

Vous devez également configurer CORS pour autoriser l'application React.
Exemple :

```
http://localhost:5173
```

8. Gestion des rôles

La présence d'un JWT valide ne doit pas donner automatiquement accès à toutes les fonctionnalités.
Vous devez contrôler les autorisations selon le rôle.

8.1. EMPLOYEE

Un employé peut :

— consulter son profil ;
— modifier son profil ;
— créer un incident ;
— consulter ses propres incidents ;
— consulter le détail de ses propres incidents.

Un employé ne peut pas :

— consulter les incidents des autres employés ;
— affecter un technicien ;
— modifier le statut d'un incident ;
— consulter la liste des utilisateurs.

8.2. TECHNICIAN

Un technicien peut :

— consulter les incidents qui lui sont affectés ;
— consulter leur détail ;
— modifier leur statut ;
— ajouter une note de traitement.

8.3. ADMIN

Un administrateur peut :

— consulter tous les incidents ;
— consulter tous les utilisateurs ;
— activer ou désactiver un utilisateur ;
— affecter un incident à un technicien ;
— consulter les statistiques globales.

9. Sécurisation selon les rôles

Travail de recherche

Recherchez comment utiliser une ou plusieurs des solutions suivantes :

```
hasRole()
hasAuthority()
@PreAuthorize
@EnableMethodSecurity
```

Par exemple :

`/api/admin/**`

doit être accessible uniquement à un utilisateur ayant le rôle ADMIN.

10. Endpoints obligatoires

10.1. Profil

Méthode	URL	Accès
GET	/api/profile	Authentifié
PUT	/api/profile	Authentifié

10.2. Incidents

Méthode	URL	Accès
POST	/api/incidents	EMPLOYEE
GET	/api/incidents/my	EMPLOYEE
GET	/api/incidents/{id}	Selon propriétaire ou rôle
GET	/api/incidents/assigned	TECHNICIAN
PUT	/api/incidents/{id}/status	TECHNICIAN ou ADMIN
PUT	/api/incidents/{id}/assign/{technicianId}	ADMIN
GET	/api/incidents	ADMIN

10.3. Administration

Méthode	URL	Accès
GET	/api/admin/users	ADMIN
PUT	/api/admin/users/{id}/enable	ADMIN
PUT	/api/admin/users/{id}/disable	ADMIN
GET	/api/admin/statistics	ADMIN

11. Règles métier

Votre application doit respecter les règles suivantes :

1. Un nouvel utilisateur reçoit automatiquement le rôle `EMPLOYEE`.
2. Un utilisateur ne peut jamais choisir le rôle ADMIN lors de son inscription.
3. Les mots de passe doivent être encodés avec BCrypt.
4. Un employé ne peut consulter que ses propres incidents.
5. Un technicien ne peut modifier que les incidents qui lui sont affectés.
6. Seul un administrateur peut affecter un technicien.
7. Un utilisateur désactivé ne peut plus se connecter.
8. Lorsqu'un incident est CLOSED, son statut ne peut plus être modifié normalement.

12. Contrôle du propriétaire d'un incident

Considérez l'URL suivante :

```
GET /api/incidents/12
```

Un employé peut essayer cette URL directement avec Postman, même si l'incident ne lui appartient pas.

Exigence de sécurité

Le backend doit vérifier que l'incident appartient réellement à l'utilisateur connecté avant de retourner les données.
Le fait de masquer un bouton ou un incident dans React ne constitue pas une protection suffisante.

Travail de recherche

Trouvez une solution permettant de récupérer l'utilisateur actuellement authentifié puis de le comparer au propriétaire de l'incident.
Vous pouvez notamment vous intéresser à :

```
Authentication
SecurityContextHolder
Principal
```

13. Validation des données

Vous devez valider les données reçues par le backend.
Exemples :

- email obligatoire et valide ;
- mot de passe d'une longueur minimale ;
- titre obligatoire ;
- description obligatoire ;
- priorité obligatoire.

Travail de recherche

Recherchez l'utilisation de :

```
@NotBlank
@Email
@Size
@Valid
```

14. Gestion des exceptions

Votre API doit retourner des réponses d'erreur propres.
Vous devez notamment gérer :

— utilisateur introuvable ;
— email déjà utilisé ;
— mauvais mot de passe ;
— incident introuvable ;
— accès interdit ;
— données invalides.

Exemple de réponse :

```
{
    "status": 404,
    "message": "Incident introuvable"
}
```

Travail de recherche

Recherchez l'utilisation de :

`@ControllerAdvice`
`@ExceptionHandler`

Recherchez également la différence entre :

`400 Bad Request`
`401 Unauthorized`
`403 Forbidden`
`404 Not Found`

15. Frontend React

Votre frontend doit contenir au minimum les éléments suivants :

```
src/
|
+-- components/
|   |
|   +-- Navbar.jsx
|   +-- ProtectedRoute.jsx
|   +-- RoleRoute.jsx
|
+-- pages/
|   |
|   +-- Login.jsx
|   +-- Register.jsx
|   +-- Dashboard.jsx
|   +-- Profile.jsx
|   +-- MyIncidents.jsx
|   +-- NewIncident.jsx
|   +-- IncidentDetails.jsx
|   +-- AssignedIncidents.jsx
```

```
|   +-- AdminUsers.jsx
|   +-- AdminIncidents.jsx
|
+-- services/
|   |
|   +-- api.js
|   +-- authService.js
|   +-- incidentService.js
|
+-- App.jsx
+-- main.jsx
```

16. Gestion du JWT côté React

Après connexion, stockez le JWT dans le navigateur.
Configurez Axios afin d'envoyer automatiquement :

```
Authorization: Bearer <JWT>
```

dans les requêtes protégées.
Utilisez pour cela un interceptor Axios.

17. Gestion de l'expiration du JWT

Configurez une durée de vie limitée du token.
Pour tester votre application, vous pouvez utiliser par exemple :

```
5 minutes
```

Lorsque le backend retourne :

```
401 Unauthorized
```

le frontend doit :

1. supprimer le JWT ;
2. rediriger l'utilisateur vers la page Login ;
3. afficher un message indiquant que sa session a expiré.

Travail de recherche

Recherchez comment utiliser :

```
axios.interceptors.response.use(...)
```

18. Protection des routes React

Créez un composant :

`ProtectedRoute`

pour empêcher un utilisateur non authentifié d'accéder aux pages protégées.
Créez également un composant :

`RoleRoute`

pour contrôler l'accès selon le rôle.
Exemple :

`/admin/users`

ne doit être accessible qu'à un ADMIN.

Important

La protection des routes React améliore l'expérience utilisateur, mais la véritable sécurité doit toujours être assurée côté backend.

19. Dashboard

Votre application doit afficher un Dashboard adapté au rôle connecté.

19.1. Dashboard EMPLOYEE

Afficher au minimum :

- nombre total de ses incidents ;
- nombre d'incidents OPEN ;
- nombre d'incidents IN_PROGRESS ;
- nombre d'incidents RESOLVED ou CLOSED ;
- ses derniers incidents.

19.2. Dashboard TECHNICIAN

Afficher :

- nombre d'incidents affectés ;
- nombre d'incidents critiques ;
- nombre d'incidents en cours ;
- nombre d'incidents résolus.

19.3. Dashboard ADMIN

Afficher :

— nombre total d'utilisateurs ;
— nombre d'employés ;
— nombre de techniciens ;
— nombre total d'incidents ;
— nombre d'incidents OPEN ;
— nombre d'incidents IN_PROGRESS ;
— nombre d'incidents CLOSED.

20. Historique des incidents

Vous devez conserver un historique des changements importants.
Exemple :

```
22/10/2026 10:20 : Incident cree
22/10/2026 11:00 : Affecte a Technicien A
22/10/2026 11:15 : OPEN -> IN_PROGRESS
22/10/2026 14:20 : IN_PROGRESS -> RESOLVED
```

Vous pouvez créer une entité :

```
IncidentHistory
```

Définissez vous-même les attributs nécessaires.

21. Notes de traitement

Un technicien doit pouvoir ajouter des notes pendant le traitement d'un incident.
Exemples :

```
"Le poste a ete isole du reseau."

"Analyse antivirus effectuee."

"Le malware a ete supprime."
```

Vous pouvez créer une entité :

```
IncidentComment
```

Définissez sa structure ainsi que sa relation avec l'incident.

22. Gestion du profil

Chaque utilisateur doit pouvoir consulter son profil.
Il doit également pouvoir modifier :

— son nom ;

— son email.

Ajoutez une fonctionnalité permettant de changer le mot de passe.
Le formulaire doit demander :

— ancien mot de passe ;

— nouveau mot de passe ;

— confirmation du nouveau mot de passe.

Le backend doit vérifier l'ancien mot de passe avant d'enregistrer le nouveau.

23. Refresh Token

Vous pouvez compléter votre système d'authentification avec un Refresh Token.
Le fonctionnement attendu est le suivant :

```
Login
  |
  +------> Access Token
  |
  +------> Refresh Token


Access Token expire
  |
  v
POST /auth/refresh
  |
  v
Nouveau Access Token
```

Travail de recherche

Recherchez :

— la différence entre Access Token et Refresh Token ;

— leurs durées de vie ;

— la manière de renouveler un Access Token ;

— le comportement attendu lors de la déconnexion.

24. Scénario de test

Votre application doit permettre de réaliser le scénario suivant :

1. Créez deux comptes EMPLOYEE.
2. Créez au moins un compte TECHNICIAN.
3. Créez un compte ADMIN.
4. Connectez-vous avec le premier employé.
5. Déclarez deux incidents.
6. Connectez-vous avec le deuxième employé.
7. Déclarez un incident.
8. Vérifiez que le premier employé ne peut pas consulter l'incident du deuxième.
9. Connectez-vous avec ADMIN.
10. Consultez tous les incidents.
11. Affectez un incident à un technicien.
12. Connectez-vous avec le technicien.
13. Consultez les incidents qui lui sont affectés.
14. Modifiez le statut :

    ```
    OPEN -> IN_PROGRESS
    ```

    puis :

    ```
    IN_PROGRESS -> RESOLVED
    ```

15. Ajoutez une note de traitement.
16. Reconnectez-vous avec l'employé.
17. Vérifiez le nouvel état de l'incident et son historique.

25. Tests de sécurité obligatoires

Vous devez tester avec Postman les situations suivantes :

1. accès à une API protégée sans JWT ;
2. accès avec un JWT invalide ;
3. accès avec un JWT expiré ;
4. EMPLOYEE essayant d'accéder à une route ADMIN ;
5. EMPLOYEE essayant d'accéder à un incident appartenant à un autre utilisateur ;
6. TECHNICIAN essayant de modifier un incident qui ne lui est pas affecté ;
7. utilisateur désactivé essayant de se connecter ;
8. inscription avec un email déjà existant ;
9. création d'un incident avec des données invalides.

26. Livrables

Vous devez rendre :

1. le projet Spring Boot ;
2. le projet React ;
3. un fichier README ;
4. un rapport PDF ;
5. un diagramme des entités ;
6. un diagramme de l'architecture générale ;
7. des captures des tests Postman ;
8. des captures de l'application ;
9. une démonstration fonctionnelle.

27. README

Votre README doit expliquer :

— comment lancer le backend ;
— comment lancer React ;
— les ports utilisés ;
— les comptes de test ;
— les rôles disponibles ;
— les principaux endpoints ;
— les fonctionnalités réalisées.